

STA 141B Homework 1 Report

Kylie Dickinson

2026-04-13

Approach

The goal is to re-format the text into a dataframe. The beginning of the file indicates comments; to ensure they will not interfere with the dataframe, lines that start with “#” will be ignored.

```
text = text[substring(text, 1, 1) != "#"]
```

To start parsing the data, a function called `parse_line` is made. After the data is read in via `readLines()`, `parse_line` first splits the line using the delimiter “;”, followed by allocating the new splits into their rightful categories:

- `timestamp` takes the “t=” split
- `id` takes the “id=” split
- `x`, `y`, and `z` takes the “pos=” split, then cut into three segments by the delimiter “,” to get individual columns
- `degree` takes the “degree=” split

```
parts = strsplit(line, ";")[[1]]

t_val = sub("t=", "", parts[1])
id_val = sub("id=", "", parts[2])

pos_vals = strsplit(sub("pos=", "", parts[3]), ",")[[1]]
x_val = pos_vals[1]
y_val = pos_vals[2]
z_val = pos_vals[3]

deg_val = sub("degree=", "", parts[4])
```

Once this is finished, the mac addresses are next to format. The approach is to have each unique mac address have its own row, with the timestamp, id, position (x, y, z), and degree being duplicated across every mac row for that line.

To obtain the mac addresses, an inner function is written inside the `parse_line` function to parse in the mac variables by ignoring the first 4 splits (by “;”). This function takes in the rest of the splits after degree, ignores empty strings, and splits on the delimiter “=”, which divides the mac address from its three properties of signal, channel, and device type. A check is done on the new split to ensure the length is 2 (accounting for the address split and the properties split), returning NULL if failed for the mac entry so that it is filtered out.

```

addr_split = strsplit(entry, "=")[[1]]
if (length(addr_split) < 2) return(NULL)

address = addr_split[1] # first item

vals = strsplit(addr_split[2], ",")[[1]] # second item
signal = ifelse(length(vals) >= 1, vals[1], NA)
channel = ifelse(length(vals) >= 2, vals[2], NA)
device_type = ifelse(length(vals) >= 3, vals[3], NA)

```

The address is taken by getting the first index, followed by the values of signal, channel, and device type as the second index, and then allocating them to their own columns. There is a check to make sure there is a value for each of the descriptors. A dataframe is constructed within this inner function that consists of the mac address and corresponding signal, channel, and device type. The inner mac-parsing function ends here, and after checking to make sure the `macs_parsed` list has no NULLs and is not empty, the `parse_line` function finishes by binding the timestamp, id, x, y, z, and degree to each of the mac addresses.

Once `parse_line` is defined, `lapply()` applies it to every line in the text file, which produces a list of dataframes. Since there is a dataframe per line, `do.call(rbind, result_list)` is used to bind them together into one dataframe called `final_df`.

```

result_list = lapply(text, parse_line)
final_df = do.call(rbind, result_list)

```

Numeric columns have their data types converted to either numeric or integer (signal, device type) and the timestamp is converted to UTC time for readability. A stack overflow [post](#) was used in the assistance of converting the timestamp.

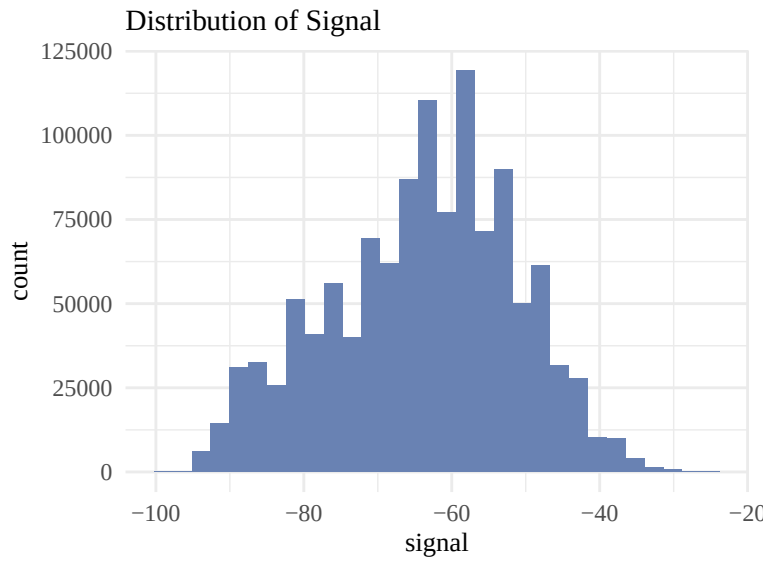
Corner cases are handled throughout `parse_line`. Comment lines beginning with “#” are filtered out and trailing semicolons that produce empty strings are removed before mac parsing. Malformed mac address entries are also eliminated via the 2-length check and return NULL. Mac descriptors (signal, channel, device type) are reviewed by checking for the three values and fills in with NA if there is a missing value, rather than throw an index error. After the inner function finishes, the `macs_parsed` list is re-checked for NULLs and if it is empty, it returns NULL and is dropped from the `mac_df = do.call(rbind, macs_parsed)` process.

Data Validation

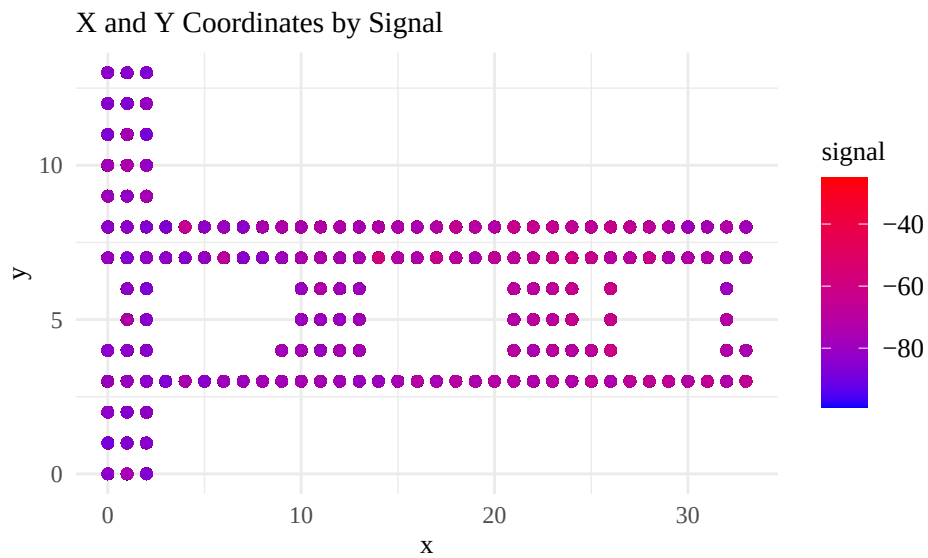
The final dataframe `final_df` is previewed as below:

timestamp	id	x	y	z	degree	mac_address	signal	channel	device_type
2006-02-11 07:31:58	00:02:2D:21:0F:33	0	0	0	0	00:14:bf:b1:97:8a	-38	2437000000	3
2006-02-11 07:31:58	00:02:2D:21:0F:33	0	0	0	0	00:14:bf:b1:97:90	-56	2427000000	3
2006-02-11 07:31:58	00:02:2D:21:0F:33	0	0	0	0	00:0fa3:39:e1:c0	-53	2462000000	3
2006-02-11 07:31:58	00:02:2D:21:0F:33	0	0	0	0	00:14:bf:b1:97:8d	-65	2442000000	3
2006-02-11 07:31:58	00:02:2D:21:0F:33	0	0	0	0	00:14:bf:b1:97:81	-65	2422000000	3
2006-02-11 07:31:58	00:02:2D:21:0F:33	0	0	0	0	00:14:bf:3b:c7:c6	-66	2432000000	3
2006-02-11 07:31:58	00:02:2D:21:0F:33	0	0	0	0	00:0fa3:39:dd:cd	-75	2412000000	3
2006-02-11 07:31:58	00:02:2D:21:0F:33	0	0	0	0	00:0fa3:39:e0:4b	-78	2462000000	3
2006-02-11 07:31:58	00:02:2D:21:0F:33	0	0	0	0	00:0fa3:39:e2:10	-87	2437000000	3
2006-02-11 07:31:58	00:02:2D:21:0F:33	0	0	0	0	02:64:fb:68:52:e6	-88	2447000000	1
2006-02-11 07:31:58	00:02:2D:21:0F:33	0	0	0	0	02:00:42:55:31:00	-84	2457000000	1

The structure and values appear consistent with the expected format from the raw data.



The signal distribution looks normal with no evident abnormalities. Signal strength is also examined against the x- and y-coordinates.

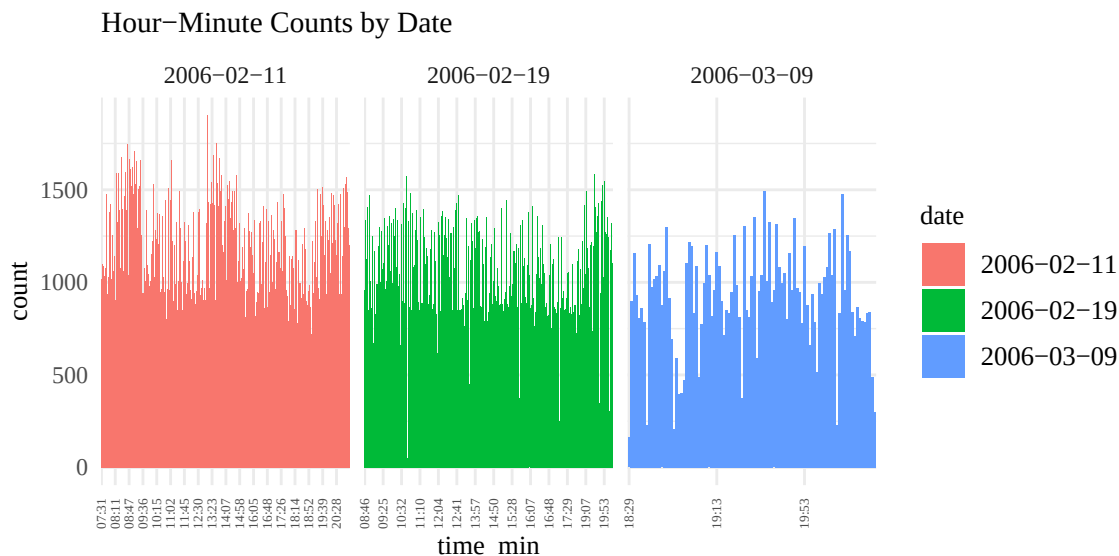


Next, the timestamp variable is examined and a sanity check was performed to ensure the dates were converted correctly.

```
date = as.Date(final_df$timestamp)
print(unique(date))
```

date
2006-02-11
2006-02-19
2006-03-09

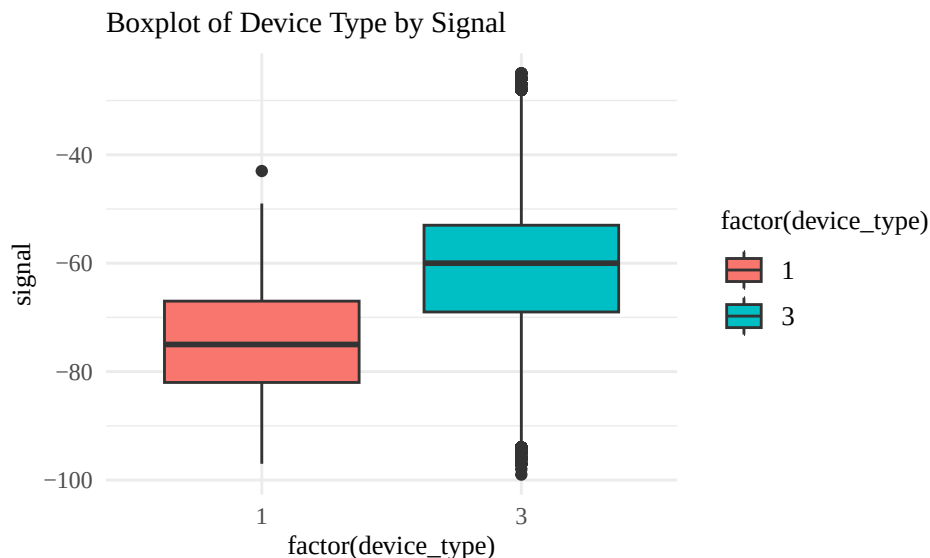
There looks to be three unique dates after conversion. Since the timestamp was converted, a time-based distribution was also plotted to spot abnormalities. The resulting plot appears consistent across all dates with no obvious anomalies.



A summary of `final_df` contains a few notable details. The device type seems to only contain two device types, 1 and 3, and the z-coordinate is 0.0 for all entries. Below is a summary of all numeric variables, with an inclusion of the timestamp.

column	min	q1	median	mean	q3	max
timestamp	2006-02-11 07:31:58.35	2006-02-11 13:28:08.67	2006-02-11 20:05:08.89	2006-02-16 15:50:07.36	2006-02-19 14:56:44.03	2006-03-09 20:41:10.46
x	0	2	12	13.732869397286	23	33
y	0	3	6	5.87618861435607	8	13
z	0	0	0	0	0	0
degree	0	90	180	167.157622703594	270	359.9
signal	-99	-73	-62	-63.8539760398366	-55	-25
channel	2412000000	2427000000	2437000000	2437960513.7996	2447000000	2472000000
device_type	1	3	3	2.65609311898499	3	3

The boxplot of signal by device type shows no obvious abnormalities, and any outliers appear reasonable.



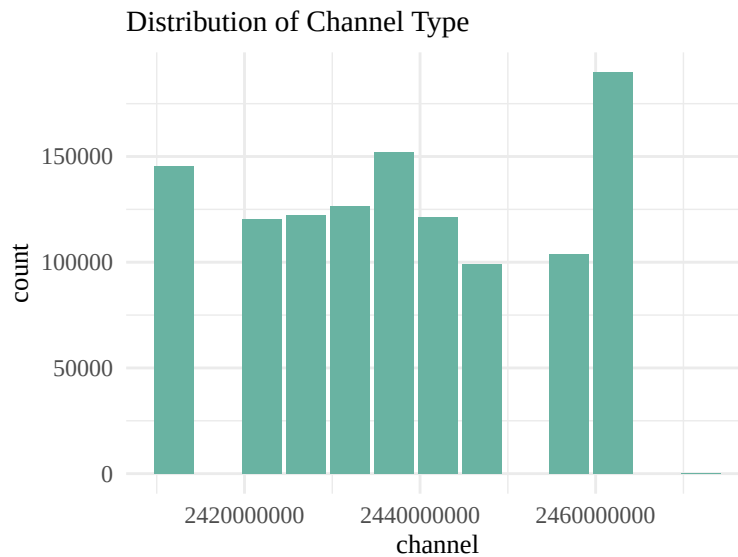
As further sanity checks, each column in `final_df` are examined for NAs, where none are found, as well as taking a look at the number of unique values in each column. In this way, it is confirmed that there are only two device types, only one value (0.0) for the z-coordinate, and a singular id (00:02:2D:21:0F:33) used for all entries. It's confirmed in another check in the validation file that there is only one id.

```
print(colSums(is.na(final_df)))
print(sapply(final_df, function(x) length(unique(x))))

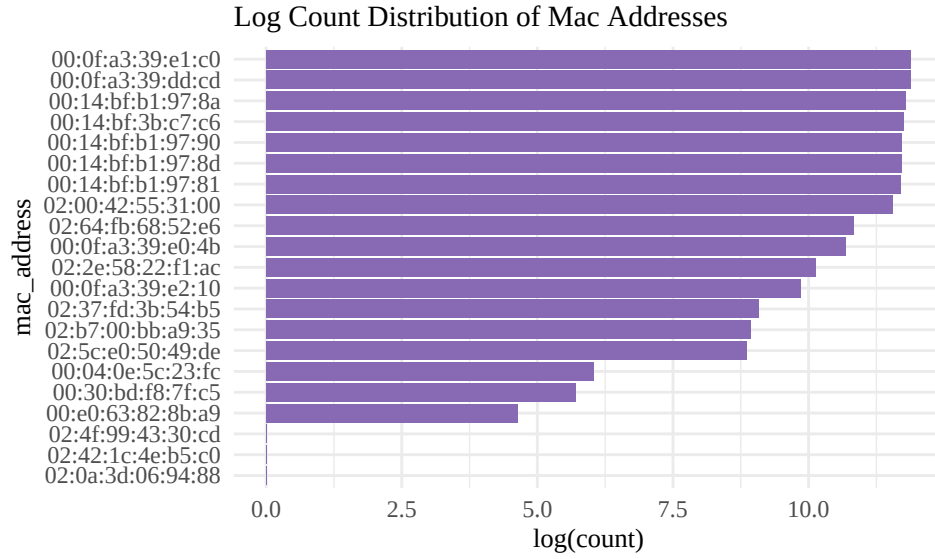
length(unique(final_df$id))
```

column	unique_values
timestamp	146074
id	1
x	34
y	14
z	1
degree	203
mac_address	21
signal	75
channel	10
device_type	2
time_min	775
date	3

For further insight, device 1 has 203185 entries (proportion=0.172) and device 3 has 978443 entries (proportion=0.828). A look at the mac address channels also shows no problem, although channel 2472000000 has a notable smaller occurrence than the other channels.



The distribution below shows reasonable values for the mac addresses except for the last three addresses, but a closer analysis shows that the three addresses have only one entry, so it minimally shows up on the graph even with a scaled x-axis.



mac_address	n
00:0f:a3:39:e1:c0	145862
00:0f:a3:39:dd:cd	145619
00:14:bf:b1:97:8a	132962
00:14:bf:3b:c7:c6	126529
00:14:bf:b1:97:90	122315
00:14:bf:b1:97:8d	121325
00:14:bf:b1:97:81	120339
02:00:42:55:31:00	103887
02:64:fb:68:52:e6	50852
00:0f:a3:39:e0:4b	43508
02:2e:58:22:f1:ac	25112
00:0f:a3:39:e2:10	19162
02:37:fd:3b:54:b5	8732
02:b7:00:bb:a9:35	7602
02:5c:e0:50:49:de	6997
00:04:0e:5c:23:fc	418
00:30:bd:f8:7f:c5	301
00:e0:63:82:8b:a9	103
02:0a:3d:06:94:88	1
02:42:1c:4e:b5:c0	1
02:4f:99:43:30:cd	1

Overall, the validation checks indicate that the dataset is clean, consistent, and free of missing values.

AI Usage

The AI used was ChatGPT. The prompts are as follows:

i have a text file that has multiple lines with a delimiter of “;” i want to read it into a csv file in R and this is my strategy: for “t=” until “;” use for timestamp for “id=” until “;” use for id for “pos=” until “;” use for position for “degree=” until “;” use for degree starting after the first four “;” (after the time, id, position, and degree), the mac address is where “00:” and “02:”

divided by semicolons * starting when seeing a “;”, if it is followed by a “00” or “02”, allocate to an array until the next “;” or “t” (to deal with the next timestamp)

i want to divide the position into x, y, z, and the pos values are divided by the delimiter “,” for the xyz coordinates. for the mac address, i want to break it into four parts: the address (where after the semicolon for the degree, until an equal sign = is what the mac address is, and then the signal is the value after the = until the delimiter “,”, the channel is after the signal), the signal is after that comma until the next comma, and the device type is the comma after until the next semicolon which denotes the next mac address. there’s the case of it also not having a semicolon after, where it would just be a new timestamp. the goal is to have each unique mac address have its own row with its traits of timestamp, id, pos, degree, signal, channel, device type as columns

i want to change how the mac addresses are grabbed. after the semicolon that follows the degree, the rest are mac addresses separated by “;” until t= comes up again at the start of a new paragraph. however, not all mac addresses start with 00: or 02:

I refined the prompt by giving more details (splitting the position into three parts instead of one), and also included the mac address in my second prompt. I also discovered that not all mac addresses start with 00: or 02:, so I removed that `grep` section of my code to broadly address the mac addresses.

The AI did not provide any errors for the code, rather, any errors along the coding process were user-related due to incorrect prompts, that were then refined over time. A highlight of how my code changed was how the mac addresses were included, where before I did not consider them, and also fixing the assumption that all mac addresses would start with 00: or 02:.

I also wanted to convert the data types from the character type (if applicable), which took some tweaking due to formatting issues. This includes utilizing the `options(scipen=999)` in my setup for tables, and understanding how to make the timestamp readable.

The final code with functions took about 184 seconds (~3 minutes).